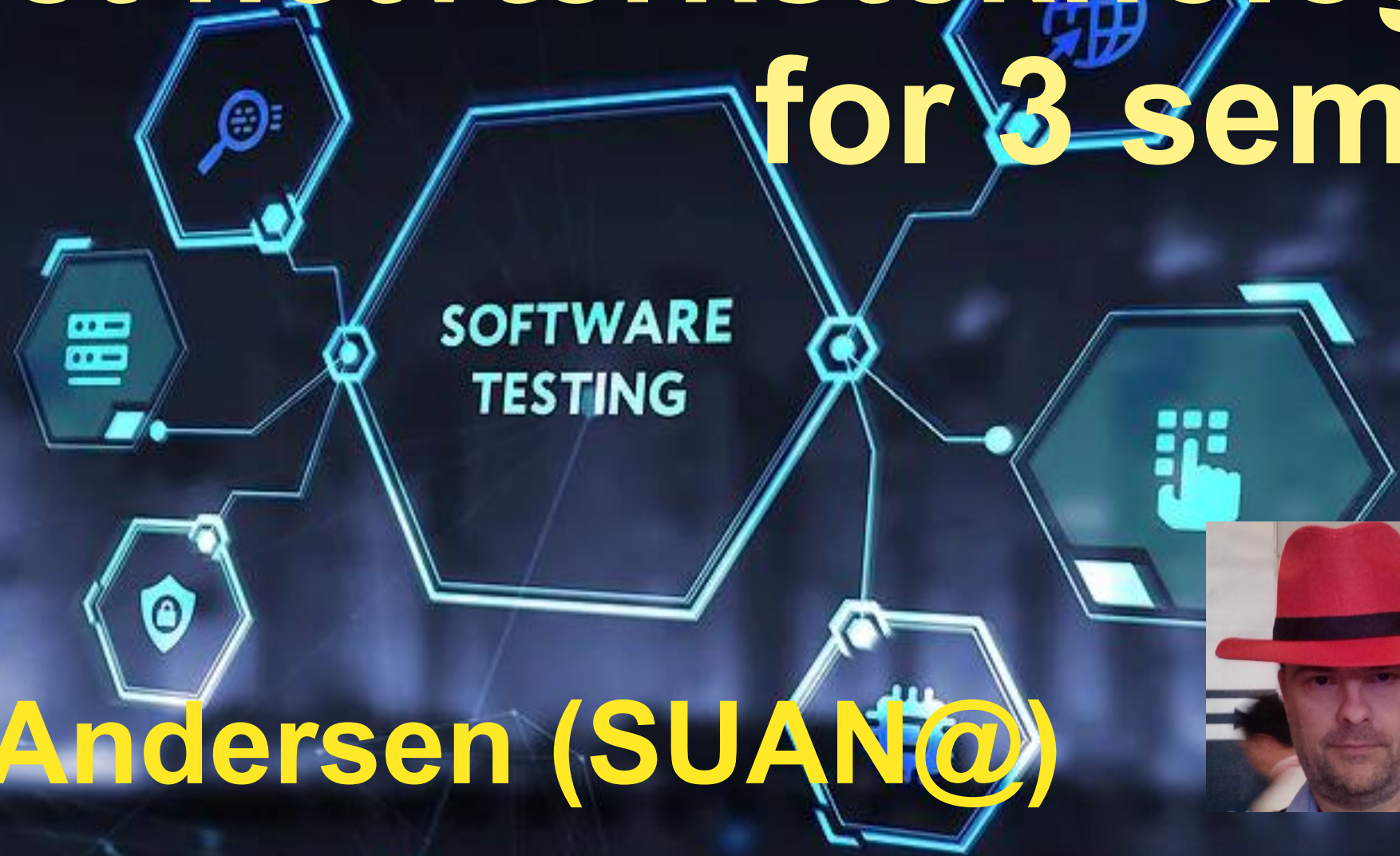


Udvidet netværksteknologi for 3 sem.



Sune Andersen (SUAN@)



I dag :

- **Hvad er test?**
- **Hvad er softwaretest og hvorfor?**
- **Vandfaldsmodellen**
- **V-Modellen**
- **Case: CrowdStrike 2024**
- **Case: Sejldroner fra Køge**
- **Case: Mars Pathfinder**
- **Dagens opgave 2 : Test zealand.dk med Playwright**



Hvad er test?

Test er ikke

- Bare afvikling af testcases
- Bare verificering af krav/specifikationer

Test **er** måling af kvalitet

- For at skabe synlighed og viden om produktet, så der kan træffes kvalificerede beslutninger på baggrund heraf.
- For at forebygge/finde fejl
- Opbygge tillid til produktet
- Verificering og validering de 2 yderpunkter i V-modellen.



Hvad er test?

1. Test viser tilstedeværelse af defekter, ikke deres fravær

En test kan vise, at der findes defekter, men kan ikke bevise, at der ingen defekter er. En test kan mindske sandsynligheden for, at der stadig findes uopdagede defekter i softwaren, men selv hvis testen ikke finder defekter, er testen ikke et bevis for, at koden er korrekt.

2. Udtømmende test er umulig

Det er umuligt at teste alle kombinationer af input og forudsætninger, selv i trivielle tilfælde. I stedet for at forsøge at teste udtømmende, bør man anvende risikoanalyse, testteknikker og prioritering for at begrænse testens fokus.

3. Tidlig test sparer både tid og penge

For at finde defekter tidligt, bør både statiske og dynamiske tests påbegyndes så tidligt som muligt i softwareudviklingens livscyklus. Tidlig test kaldes også shift left. En test tidligt i softwareudviklingens livscyklus hjælper til at mindske eller undgå dyre ændringer (se afsnit 3.1)

4. Defekter samler sig i klynger

Et lille antal moduler indeholder normalt de fleste af de defekter som man finder i løbet af tests før releasen, eller er ansvarlige for de fleste driftsafvigelser. Det er vigtigt at medregne de forventede klynger af defekter og de observerede klynger af defekter i test eller drift i den risikoanalyse, der skal anvendes til at indsnævre testens fokus (som nævnt i princip 2).

5. Pas på pesticidparadokset

Hvis samme test gentages igen og igen vil den med tiden ikke finde nye defekter. For at finde nye defekter er der behov for at ændre de eksisterende tests og testdata, og at udarbejde nye tests. (Tests, der ikke længere er i stand til at finde defekter, ligesom pesticider med tiden mister evnen til at dræbe insekter.) I nogle tilfælde, f.eks. ved automatiseret regressionstest, har pesticidparadokset et fordelagtigt udfald, hvilket er det relativt lave antal regressionsdefekter.

6. Test er afhængig af konteksten

En test udføres forskelligt alt efter konteksten. F.eks. skal sikkerhedskritisk industriel styringssoftware testes anderledes end en e-commerce app. Som et andet eksempel skal test afvikles på én måde i et agilt projekt og på en anden måde i et sekventielt softwareudviklingscyklusprojekt (se afsnit 2.1)

7. Fravær-af-fejl-fejltagelsen

Nogle organisationer forventer, at testere kan gennemføre alle mulige tests og finde alle tilstedeværende defekter, men ud fra henholdsvis princip 2 og 1 ved vi, at det ikke er muligt. Desuden er det fejlagtigt at forvente, at blot man finder og udbedrer et stort antal af defekter, så sikrer det, at systemet er vellykket. F.eks. kan man vha. grundig test af alle specificerede krav og at udbedre alle defekter stadig fremstille et system, der er svært at anvende, som ikke lever op til brugernes behov og forventninger eller som er underlegent til sammenligning med konkurrerende systemer.



Kilde : <https://medarbejdere.au.dk/strategi/itgovernance/test/mere-viden-om-test/>

• Hvad er softwaretest og hvorfor?

ISTQB = International Software Testing Qualifications Board

- Fælles terminologi inden for softwaretest
- Ensartet måde at gøre ting på
- En professionel tilgang til test af software
- Viden om international best practice

...og det, vi baserer vores testmodel på.

Den korte version - Vi tester for at:

- ✓ finde fejl før softwaren kommer i brug og kan gøre skade.
- ✓ kontrollere krav og øge kvalitet
- ✓ beskytte brugeren mod utilsigtede brug (Brugeren eller hackere)
- ✓ skabe tillid til systemet/softwaren

Vil man læse mere er [ISO/IEC/IEEE 29119](#) stedet at starte ;-)



Vandfaldsmodellen

Vandfaldsmodellen

En **lineær og struktureret** model til softwareudvikling.

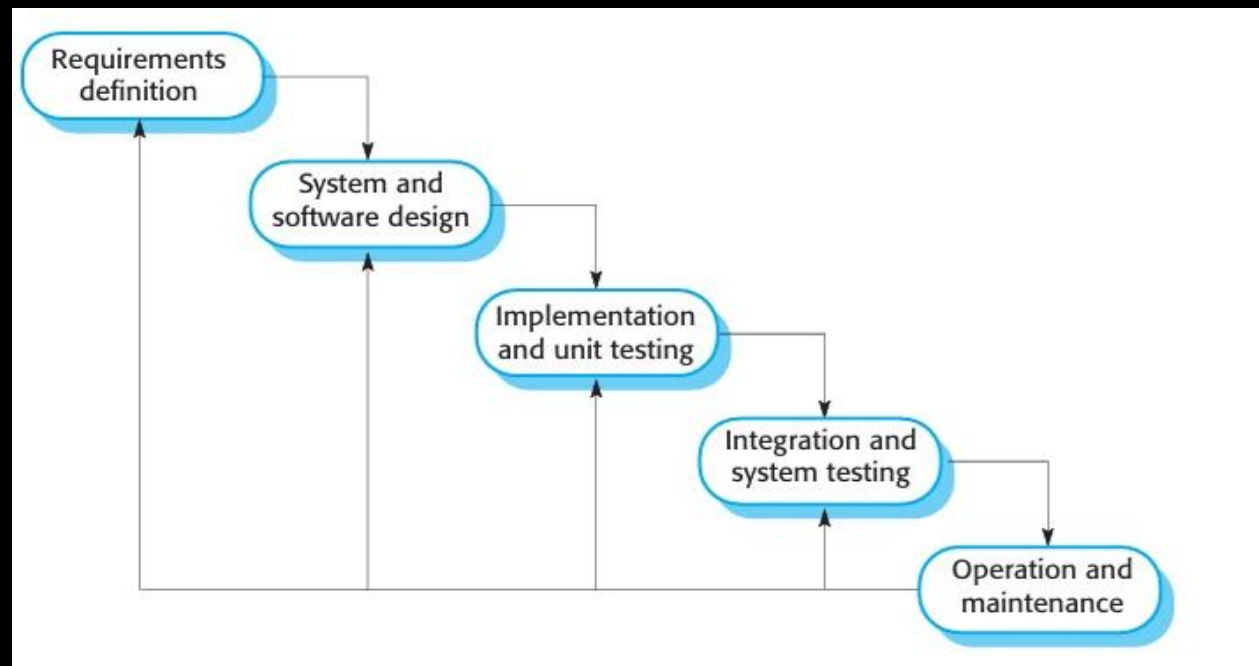
Udviklingen foregår i faste faser: **krav** → **design** → **implementering** → **test** → **drift**.

Hver fase afsluttes, før man går videre til den næste.

Fordel: Giver et klart overblik og gør processen nem at planlægge.

Ulempe: Det er svært og dyrt at ændre krav senere i processen.

Modellen egner sig bedst til projekter, hvor **kravene er kendte og stabile fra starten**.

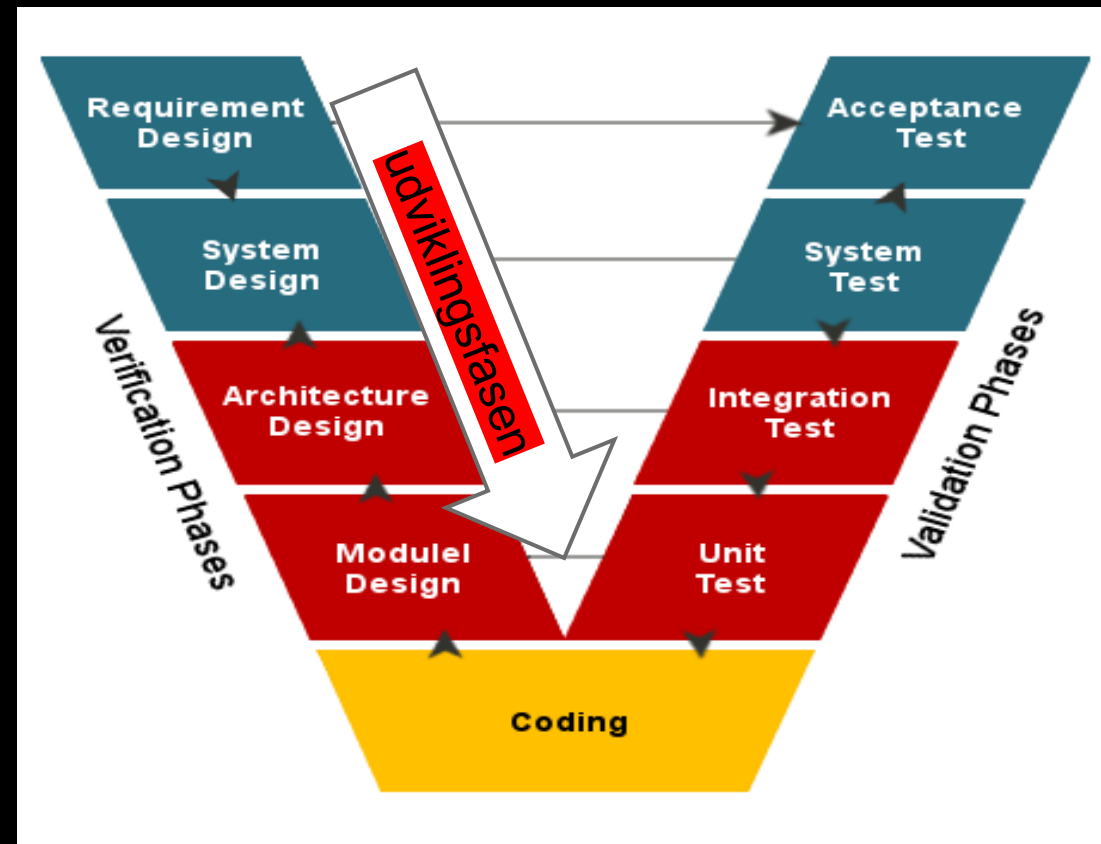


Kilde : Software engineering tenth edition af Ian Sommerville

V-Modellen

V-modellen – udviklingsfaser

- **Requirement Design:**
Fastlægger systemets krav og behov
- **System Design:** Planlægger systemets overordnede opbygning
- **Architecture Design:** Opdeler systemet i hovedkomponenter
- **Module Design:** Beskriver de enkelte moduler i detaljer
- **Coding:** Programmering af systemet



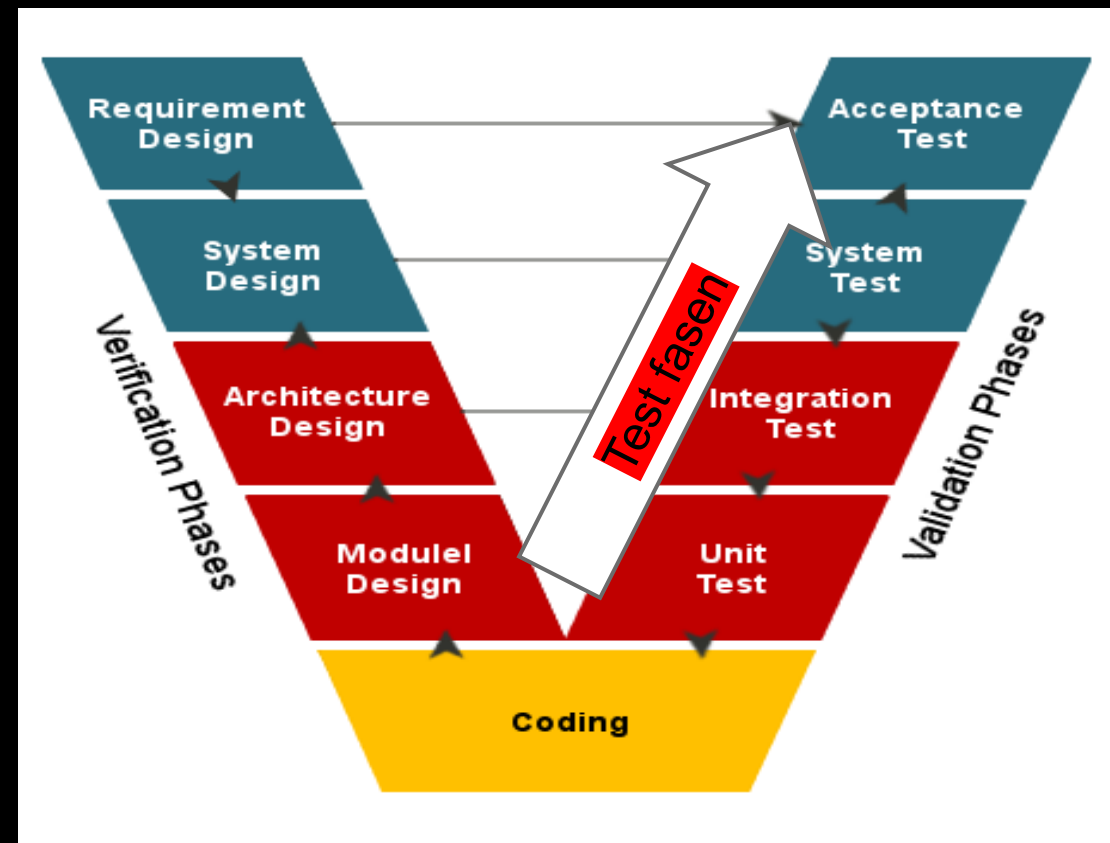
V-Modellen

V-modellen – testfaser

- **Unit Test:** Tester de enkelte moduler
- **Integration Test:** Tester om modulerne fungerer sammen
- **System Test:** Tester hele systemet samlet
- **Acceptance Test:** Tester om systemet opfylder kundens krav

Hovedidé

- **Verification:** *Bygger vi systemet rigtigt?*
- **Validation:** *Bygger vi det rigtige system?*
- Hver udviklingsfase har en **tilsvarende testfase**.



CASE : CrowdStrike 2024

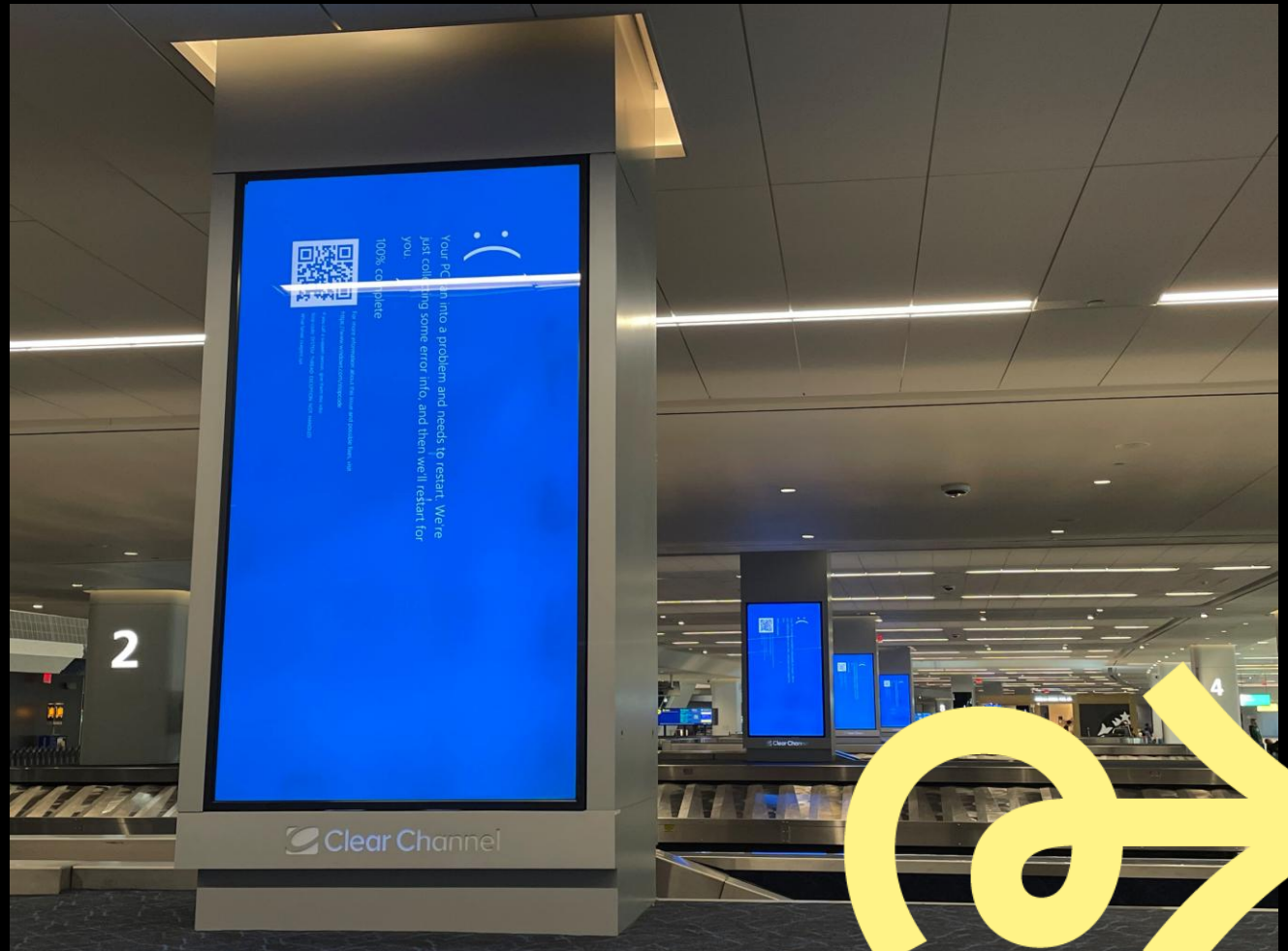
- Computeren fryser.
- Programmet lukker.
- Noget virker pludselig ikke.

Problemet ramte Windows-systemer verden over.

Flyselskaber, virksomheder, hospitaler og andre organisationer blev påvirket.

Hvad tror I er sket??

- Ca. 8,5 millioner Windows-enheder blev påvirket



CASE : CrowdStrike 2024

19. juli 2024

En softwareopdatering går galt hos Sikkerhedsfirmaet CrowdStrike(Falcon) og mange Windows-computere viser Blue Screen of Death.

Og så et stort tal: **\$5,4 mia.**
estimerede tab for Fortune 500 virksomheder

Kunne det være undgået med bedre test?



Kilde : <https://www.eccouncil.org/cybersecurity-exchange/incident-handling/crowdstrike-incident/>

CASE : Sejldroner fra Køge

16. juni 2025

Fra **Køge Marina** blev to autonome sejldroner sat i vandet.

Ingen kaptajn ombord.

Ingen besætning.

Software og sensorer hjælper med at styre fartøjet.

Forestil dig, at dronen skal:

- navigere
- undgå andre fartøjer
- følge en rute
- indsamle data
- sende information hjem

Hvad sker der, hvis én af de ting ikke virker?



Kilde : <https://www.fmi.dk/da/nyheder/2025/saildrone-sat-i-soen--forsvaret-tester-ny-kurs-for-maritim-overvagning>

CASE : Mars Pathfinder

Mars Pathfinder lander på Mars i 1997.

Alt ser ud til at fungere.

Men så begynder computeren at **resette**.

NASA kan ikke bare:

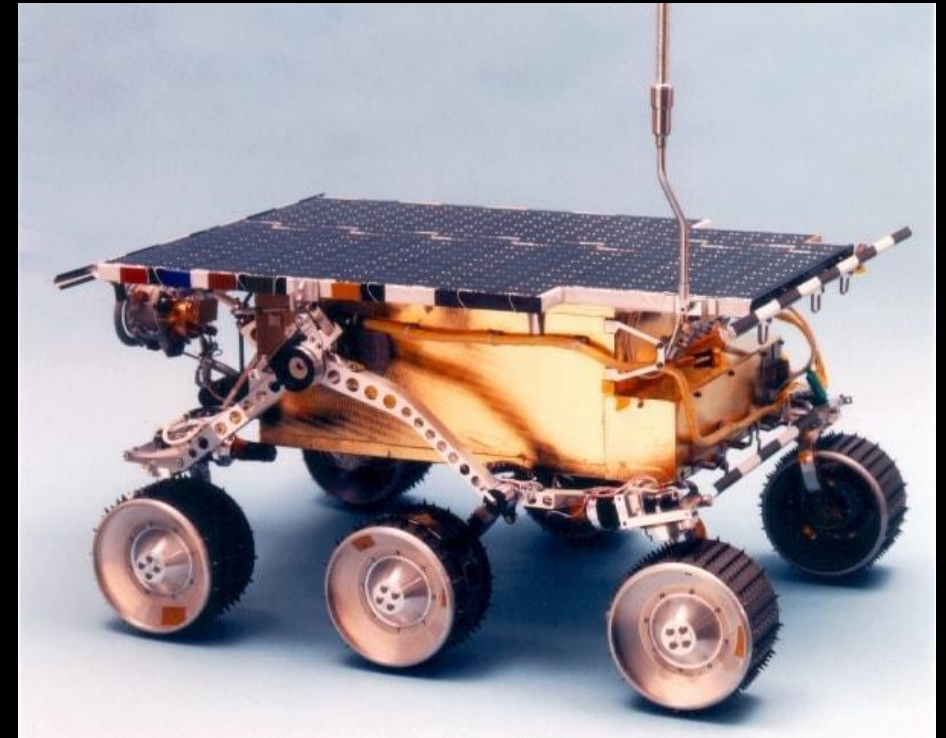
"Gå hen til computeren og genstarte den."

Der er en enorm kommunikationsforsinkelse mellem Jorden og Mars. Afhængigt af afstanden kan radiosignaler bruge omkring **22 minutter én vej**, så en kommando og et svar kan i værste fald tage omkring **44 minutter tilsammen**.

Roverens system oplevede en softwarefejl relateret til **priority inversion**.

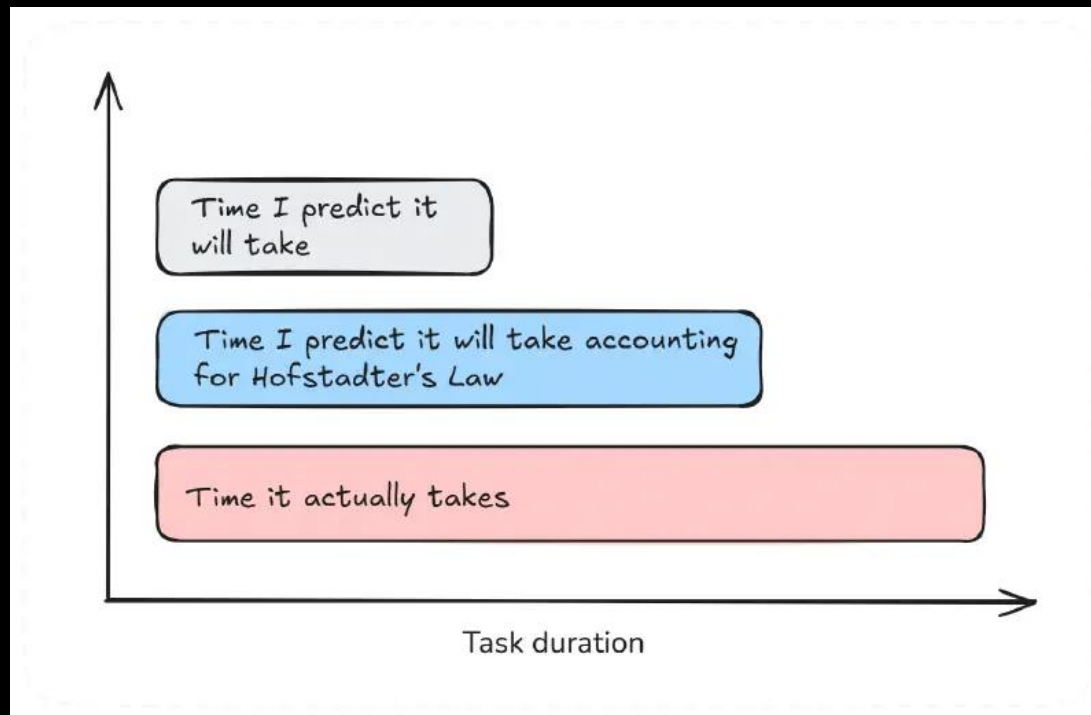
Det interessante er, at **NASA** faktisk kunne **reproducere problemet på Jorden** og finde årsagen.

"Hvordan ville I fejlfinde en computer, hvis den stod på Mars?"



Kilde : <https://postmortem.io/incidents/nasa--1997-07-04--mars-pathfinder-priority-inversion/>

- Sidste gang Opgave 1 : udviklingsmiljø
- I dag Opgave 2 : Test zealand.dk med Playwright



Nysgerrig?
Spørg løs!

Min Mail
suan@zealand.dk

